

InsurMinds

Plataforma Inteligente para Análise e Comparação de Apólices D&O

Relatório Técnico — Projeto Final I2A2 Versão 1.0 — outubro de 2026

Grupo: LatinTech

Integrantes: Fábio Castro; Lucas Godois; Pedro Campos; Rogério Sousa; Fernando Gonçalves.

Repositório: <https://github.com/RogérioSousa123/insurminds-projeto-final-latintech>

Aplicação: <https://latintech.latin-re.com>

1. Resumo executivo

A InsurMinds é um protótipo funcional para extração, organização, consulta e comparação de apólices de seguro D&O. A solução recebe documentos em PDF ou imagem, preserva o texto por página, utiliza inteligência artificial generativa para identificar condições relevantes e apresenta uma matriz comparativa rastreável.

O princípio central é a explicabilidade: um campo somente é aceito quando contém uma página válida e um trecho que pode ser confirmado no texto extraído. Dessa forma, o usuário não recebe apenas uma conclusão, mas também a evidência necessária para verificá-la.

O sistema diferencia explicitamente quatro situações: informação encontrada, informação não localizada, informação ambígua e condição não aplicável. Essa separação impede que a ausência de uma informação no processamento seja interpretada automaticamente como ausência de cobertura.

O MVP foi implementado em Python com interface Streamlit, extração de PDF por PyMuPDF, OCR Tesseract, modelos Pydantic, armazenamento SQLite e integração conversacional configurável com Anthropic ou OpenAI. Testes automatizados cobrem os principais controles de integridade.

2. Problema e contexto

Apólices D&O contêm definições, coberturas, exclusões, limites, franquias, condições temporais e regras jurisdicionais distribuídas ao longo de documentos extensos. A comparação manual exige leitura especializada, normalização de terminologia e conferência de diferentes páginas.

Uma abordagem que apenas envia dois documentos a um chatbot produz respostas de difícil auditoria e sujeitas a omissões. O problema técnico não é somente resumir texto, mas criar um processo controlado que preserve origem, diferencie certeza de ausência e converta linguagem jurídica em uma estrutura comparável.

3. Objetivos

3.1 Objetivo geral

Desenvolver uma plataforma baseada em IA generativa capaz de apoiar a análise e a comparação de pelo menos duas apólices D&O.

3.2 Objetivos específicos

- receber PDFs e imagens por uma interface simples;
- extrair texto de documentos nativos e digitalizados;
- identificar 26 critérios relevantes de D&O;
- armazenar valores, estados, páginas, trechos e confiança;
- comparar objetivamente dados estruturados;
- explicar diferenças em linguagem clara;
- responder perguntas utilizando somente evidências selecionadas;
- exportar resultados em formatos reutilizáveis;
- documentar decisões, limitações e possibilidades de evolução.

4. Escopo do MVP

O MVP analisa até quatro apólices por comparação, com limite configurável de 150 páginas e 50 MB por arquivo. Os formatos aceitos são PDF, PNG, JPEG, TIFF e WebP.

O escopo funcional inclui ingestão, extração, OCR, análise generativa, validação de evidências, persistência, visualização individual, matriz comparativa, resumo executivo, copiloto fundamentado e exportação em PDF, CSV e JSON.

Não fazem parte do escopo autenticação, gestão de organizações, integração com seguradoras, alta disponibilidade, assinatura digital, classificação regulatória definitiva ou decisão automatizada de contratação.

5. Atendimento aos requisitos

Requisito · Implementação · Evidência no sistema

Ler PDF ou imagem · PyMuPDF, Pillow e Tesseract · aba Documentos

Extrair automaticamente · fluxo por lotes e agente extrator · progresso e análise individual

Estruturar informações · taxonomia fechada e Pydantic · tabela de 26 critérios

Comparar ao menos duas apólices · comparador determinístico · aba Comparação

Mostrar principais diferenças · matriz, atenção e resumo · métricas e semáforo

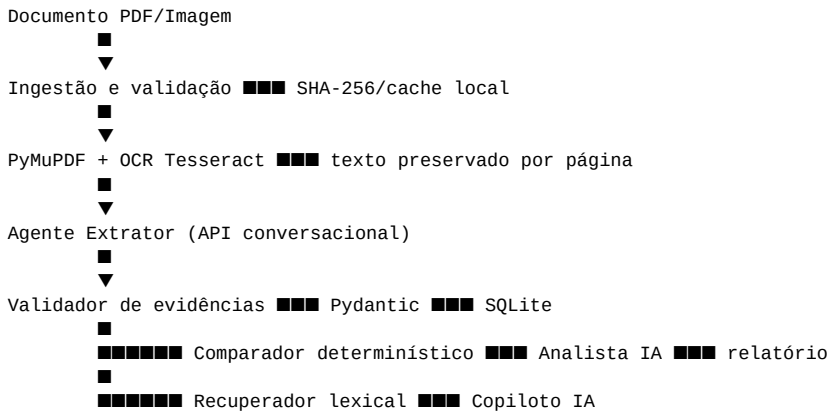
Usar IA generativa · adaptadores Anthropic/OpenAI · modelo e tokens exibidos

Disponibilizar interface · aplicação Streamlit · cinco abas funcionais

Armazenar dados · SQLite local · histórico após reinício

Consultar informações · copiloto com evidências · aba Copiloto

6. Arquitetura



A solução utiliza arquitetura modular em camadas. Modelos de domínio não conhecem a interface nem o fornecedor da IA. Serviços executam o fluxo de negócio. Adaptadores isolam Anthropic e OpenAI. O repositório encapsula o SQLite. O Streamlit coordena apenas interação e apresentação.

7. Tecnologias utilizadas

Python 3.11

Foi escolhido pelo ecossistema de processamento documental, IA, ciência de dados e prototipação. A linguagem permite manter todo o fluxo em uma única base compreensível pelo grupo.

Streamlit

Fornece upload, estado de sessão, componentes de progresso, tabelas e downloads com baixo esforço de interface. É adequado ao objetivo didático do MVP.

PyMuPDF

Extrai texto de PDFs preservando a separação por páginas e também renderiza páginas para OCR quando a camada textual é insuficiente.

Tesseract e Pillow

O Tesseract executa OCR local em português e inglês. Pillow valida, corrige orientação e converte imagens. Essa combinação evita contratar um segundo serviço externo apenas para OCR.

API conversacional de IA generativa

A interface ChatProvider possui adaptadores para Anthropic e OpenAI. A aplicação envia texto extraído e instruções, recebendo JSON solicitado por prompt. O modelo e o fornecedor são configurados no ambiente.

Pydantic

Valida tipos, intervalos de confiança, páginas positivas, estados e contratos internos. Respostas da IA nunca são persistidas diretamente sem validação.

SQLite

Atende ao volume do protótipo sem servidor adicional. Os registros completos são preservados em JSON e indexados por hash, modelo e versão do prompt.

Pandas, ReportLab e python-pptx

Pandas monta matrizes e exportações tabulares. ReportLab gera relatórios PDF. python-pptx gera o pitch deck acadêmico.

8. Agentes desenvolvidos

8.1 Agente Leitor

Valida arquivo, tamanho, criptografia e quantidade de páginas. Decide entre extração nativa e OCR e devolve uma coleção de páginas identificadas.

8.2 Agente Extrator

Recebe lotes de páginas e uma taxonomia fechada. O prompt determina que o documento é dado não confiável, proíbe conhecimento externo, exige JSON e solicita somente campos localizados no lote.

8.3 Agente Auditor

Confere se a chave pertence à taxonomia, se a página foi fornecida ao modelo e se as palavras da evidência existem na página original. Evidências sem aderência são descartadas. Aderência parcial reduz a confiança.

8.4 Agente Comparador

O núcleo do comparador é determinístico: confronta estados e valores normalizados. A IA recebe apenas uma representação reduzida das diferenças para redigir o resumo e explicar impactos sem recomendar contratação.

8.5 Agente Relator

Organiza resumo, matriz, evidências e ressalvas em PDF. As mesmas informações podem ser baixadas em CSV e JSON para auditoria ou reutilização.

8.6 Agente Copiloto

Calcula relevância lexical entre pergunta e campos extraídos, seleciona evidências e solicita uma resposta com citação no formato arquivo e página. Quando o contexto é insuficiente, orienta declarar a informação como não localizada.

9. Taxonomia e modelo de dados

A taxonomia contém 26 critérios distribuídos em:

- identificação;
- condições gerais;
- limites e franquias;
- coberturas A, B e C;
- custos de defesa e coberturas específicas;
- exclusões;
- temporalidade;

- território e jurisdição;
- eventos societários.

Cada campo possui chave estável, rótulo, categoria, status, valor legível, valor normalizado, unidade, resumo, evidências e confiança. Uma evidência contém página, trecho e confiança.

O valor legível preserva a forma contratual; o normalizado permite comparação. Por exemplo, “R\$ 10.000.000,00” pode ser armazenado como 10000000 com unidade BRL.

10. Fluxo completo de processamento

- O usuário envia um ou mais arquivos.
- A ingestão sanitiza o nome e valida formato e tamanho.
- É calculado o SHA-256 e consultado o cache pelo modelo e versão de prompt.
- Cada página de PDF é lida pelo PyMuPDF.
- Páginas com menos de 80 caracteres ou baixa qualidade seguem para OCR.
- As páginas são agrupadas sem que uma página seja cortada.
- Cada lote é enviado à API conversacional.
- A resposta é convertida para JSON; uma tentativa de reparo é feita se necessário.
- Chaves desconhecidas, páginas inválidas e trechos não confirmados são descartados.
- Candidatos duplicados são consolidados; valores conflitantes geram ambiguidade.
- Critérios ausentes são preenchidos explicitamente como não localizados.
- A análise e as páginas são persistidas no SQLite.
- O comparador confronta estados e valores normalizados.
- A IA pode gerar explicação executiva a partir da matriz estruturada.
- O usuário consulta evidências, conversa com o copiloto ou exporta o resultado.

11. Estratégia de prompts e redução de alucinações

Os prompts seguem cinco princípios:

- papel e tarefa específicos;
- taxonomia permitida e formato explícito;
- conteúdo do documento delimitado;
- proibição de inferir ou usar conhecimento externo;
- exigência de evidência literal e página.

Além do prompt, controles fora do modelo verificam a resposta. Isso é importante porque instruções sozinhas não garantem fidelidade. Um campo sem evidência confirmável deixa de entrar na base, mesmo que tenha sido apresentado com confiança alta pela IA.

12. Comparação e semáforo de atenção

O sistema não atribui uma nota absoluta às apólices. Cada critério recebe:

- atenção baixa quando os valores estruturados coincidem;
- atenção média ou alta quando diferem, conforme a relevância configurada;
- atenção indeterminada quando alguma informação está ausente ou ambígua.

O semáforo orienta revisão e não representa recomendação de compra. Cláusulas dependem do contexto do segurado e da interpretação profissional.

13. Interface e experiência do usuário

A interface possui cinco áreas:

- **Documentos:** upload, cache e progresso por etapa;
- **Apólices:** métricas, 26 critérios, evidências e avisos;
- **Comparação:** matriz, semáforo, resumo e downloads;
- **Copiloto:** perguntas fundamentadas nos documentos selecionados;
- **Como funciona:** explicação da arquitetura incorporada ao protótipo.

O modelo, consumo de tokens, páginas OCR e duração ficam visíveis para facilitar demonstração técnica.

14. Segurança, privacidade e governança

As chaves de API ficam em `.env`, arquivo ignorado pelo Git. Elas não são enviadas à interface nem gravadas no SQLite. Arquivos são processados em memória, e o MVP persiste texto e evidências para permitir consulta posterior.

Documentos contratuais podem conter dados pessoais ou confidenciais. Por isso, a demonstração deve utilizar documentos públicos, sintéticos ou autorizados. Uma implantação real exigiria política de retenção, criptografia, autenticação, autorização, registro de consentimento e avaliação jurídica da transferência de dados ao provedor de IA.

O conteúdo documental pode conter prompt injection. Os prompts declaram o documento como dado não confiável e instruem o modelo a ignorar comandos nele contidos. A taxonomia fechada e a validação posterior limitam o impacto, embora não eliminem todos os riscos.

15. Tratamento de erros

- arquivos vazios, grandes, não suportados ou protegidos geram mensagem específica;
- falhas transitórias da API recebem até três tentativas com espera progressiva;
- JSON malformatado recebe uma tentativa controlada de reparo;
- páginas sem texto registram aviso;
- OCR ausente apresenta orientação ao usuário;
- campos inválidos não interrompem toda a análise;
- SQLite utiliza transações curtas e modo WAL;
- análises concluídas permanecem disponíveis se a API ficar indisponível.

16. Testes e resultados

A suíte automatizada contém 14 testes e foi executada com sucesso no ambiente de desenvolvimento.

Os cenários cobrem:

- parsing de JSON simples e cercado por Markdown;
- rejeição de respostas que não são objetos;
- aceitação de campo permitido com evidência válida;
- descarte de chave inventada pela IA;
- descarte de evidência inexistente;

- continuidade da análise quando um lote devolve JSON inválido;
- consolidação de cláusulas narrativas sem falsa ambiguidade;
- detecção de diferença em limite monetário;
- seleção de contexto com as duas apólices no copiloto;
- normalização e validação de citações em múltiplas páginas;
- persistência e recuperação no SQLite;
- extração de texto de PDF;
- geração de relatório PDF válido, inclusive com textos longos.

Os documentos da pasta `exampLes` são sintéticos e servem para teste funcional. Eles não constituem contratos, condições de seguradora nem orientação de cobertura.

17. Justificativas das principais decisões

Componentes especializados em vez de um agente autônomo único

O fluxo possui etapas previsíveis. Módulos com responsabilidades claras são mais fáceis de testar, demonstrar e manter do que um agente com autonomia ampla.

SQLite em vez de banco vetorial

A comparação usa uma taxonomia pequena e fixa. Um banco vetorial aumentaria infraestrutura sem benefício proporcional no MVP. A recuperação lexical é suficiente para a demonstração e pode ser substituída posteriormente.

OCR local

Reduz dependências externas e deixa claro qual tecnologia resolve cada etapa. Em uma evolução comercial, serviços de Document AI poderiam melhorar tabelas e formulários.

Comparação em código

Números e estados devem produzir o mesmo resultado a cada execução. A IA é reservada à interpretação textual, onde agrega maior valor.

Evidência obrigatória

A rastreabilidade é mais importante do que maximizar quantidade de campos. É preferível marcar um item como não localizado a apresentar uma conclusão sem fonte.

18. Limitações conhecidas

- OCR depende de resolução, rotação e pacote de idioma;
- a correspondência lexical não detecta toda paráfrase válida;
- valores jurídicos complexos podem ser reduzidos de forma imperfeita;
- a taxonomia não cobre todas as extensões ou endossos do mercado;
- o sistema não possui autenticação nem isolamento entre usuários;
- o SQLite não foi projetado para grande concorrência;
- a qualidade e o custo variam conforme o modelo escolhido;
- o copiloto não utiliza embeddings nem reranker;
- a solução não determina validade jurídica nem substitui especialista.

19. Evoluções futuras

Curto prazo

- tela de revisão humana e edição controlada;
- indicadores de custo por documento;
- avaliação com conjunto rotulado;
- normalização monetária mais abrangente;
- comparação de endossos com a apólice-base.

Médio prazo

- embeddings e busca híbrida;
- processamento assíncrono e fila;
- PostgreSQL e armazenamento de objetos;
- autenticação, perfis e trilha de auditoria;
- taxonomias para E&O, Cyber e RC Geral;
- exportação para planilhas padronizadas de corretoras.

Longo prazo

- implantação privada com políticas corporativas de retenção;
- integração com portais e sistemas de gestão de seguros;
- aprendizagem a partir das correções de especialistas;
- avaliação contínua de modelos e detecção de regressões;
- motor configurável de requisitos por perfil de risco.

20. Instalação e reprodução

- instalar Python 3.11;
- criar um ambiente virtual;
- instalar `requirements.txt`;
- copiar `.env.example` para `.env`;
- configurar provedor, modelo e chave;
- instalar Tesseract para documentos digitalizados;
- executar `streamlit run app.py`;
- rodar `python -m pytest` antes da apresentação.

As instruções completas estão no README do repositório.

21. Fontes de dados

Os dois documentos distribuídos em `examples` foram redigidos exclusivamente para demonstração do projeto. São sintéticos, não reproduzem uma apólice comercial e não possuem validade contratual. Por serem criações originais do grupo, não dependem de fonte externa e permitem demonstrar o sistema sem expor dados pessoais ou contratuais.

Documentos públicos podem ser usados em validações adicionais, mas não são redistribuídos no repositório. Sempre que um documento externo for exibido na apresentação, seu título, entidade responsável, URL e data de acesso deverão ser informados.

22. Conclusão

A InsurMinds demonstra a integração de OCR, IA generativa, agentes especializados, banco de dados, automação e interface de consulta em um fluxo coerente. A arquitetura prioriza separação de responsabilidades, evidência, validação e transparência.

O resultado é um MVP simples o suficiente para ser compreendido pelo grupo e completo o suficiente para processar documentos, estruturar informações, comparar apólices e demonstrar valor ao usuário. A principal contribuição não é substituir o especialista, mas transformar leitura dispersa em uma análise verificável e reutilizável.